

Laboratório de Computação Distribuída

Prof Alcides / Prof Mario

Nome: Thais Ferreira Canguçu - 10403283

Objetivo

Construir um cluster distribuído utilizando Docker e Ubuntu, configurar SSH entre nós e executar programas MPI de forma distribuída.

Ao final, você deverá compreender:

- Diferença entre execução local e distribuída
- Conceito de master e workers
- Uso de hostfile
- Comunicação via SSH no MPI

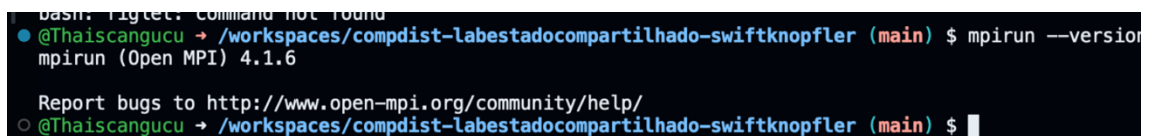
PARTE 1 – MPI local (1 máquina) 1 - Criar o Codespace 2 - Instalar o MPI

No terminal do Codespace digite:

```
sudo apt update
sudo apt install -y openmpi-bin libopenmpi-dev
figlet
```

Para verificar:

```
mpirun --version
```



```
bash: figlet: Command not found
@Thaiscanguçu → /workspaces/compdist-labestadocompartilhado-swiftnopfler (main) $ mpirun --version
mpirun (Open MPI) 4.1.6
Report bugs to http://www.open-mpi.org/community/help/
@Thaiscanguçu → /workspaces/compdist-labestadocompartilhado-swiftnopfler (main) $
```

3 - Testes locais (no host principal do Codespaces)

Teste 1 – hostname

```
mpirun -np 4 hostname
```

```
@Thaiscangucu → /workspaces/compdist-labestadocompartilhado-swiftnopfler (main) $ mpirun -np 4 hostname
There are not enough slots available in the system to satisfy the 4
slots that were requested by the application:

hostname

Either request fewer slots for your application, or make more slots
available for use.

A "slot" is the Open MPI term for an allocatable unit where we can
launch a process. The number of slots available are defined by the
environment in which Open MPI processes are run:

1. Hostfile, via "slots=N" clauses (N defaults to number of
   processor cores if not provided)
2. The --host command line parameter, via a ":N" suffix on the
   hostname (N defaults to 1 if not provided)
3. Resource manager (e.g., SLURM, PBS/Torque, LSF, etc.)
4. If none of a hostfile, the --host command line parameter, or an
   RM is present, Open MPI defaults to the number of processor cores

In all the above cases, if you want Open MPI to default to the number
of hardware threads instead of the number of processor cores, use the
--use-hwthread-cpus option.

Alternatively, you can use the --oversubscribe option to ignore the
number of available slots when deciding the number of processes to
launch.
```

Este comando solicita 4 processos, eles rodaram?

Não, pois não existem slots suficientes.

Tente reduzir para 2 e depois 1, qual rodou? Por quê?

```
@Thaiscangucu → /workspaces/compdist-labestadocompartilhado-swiftnopfler (main) $ mpirun -np 1 hostname
codespaces-0cae63
```

Agora liste quantas CPUs e threads tem a máquina do Codespaces?

`lscpu` ou `nproc`

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          48 bits physical, 48 bits virtual
Byte Order:            Little Endian
CPU(s):                 2
On-line CPU(s) list:   0,1
```

Provavelmente `nproc` trará 2 CPUs, `lscpu` trará CPU(s): 2;
Thread(s) per core: 2 e Core(s) per socket: 1, analise por que não rodou
o comando com 2 processos?

**Apesar de 2 CPUs e 4 threads, se o arquivo de configuração do MPI
achar que você só tem 1 "slot" disponível, ele vai travar a execução
por segurança para não sobrecarregar a CPU.**

Agora teste com o `--oversubscribe`

```
mpirun --oversubscribe -np 4 hostname
```

```
@Thaiscangucu → /workspaces/compdist-labestadocompartilhado-swiftnopfler (main) $ mpirun --oversubscribe  
-np 4 hostname  
codespaces-0cae63  
codespaces-0cae63  
codespaces-0cae63  
codespaces-0cae63
```

Teste 2 – Execução concorrente

```
mpirun --oversubscribe -np 4 bash -c 'echo "Olá!  
Eu sou o processo $$"; sleep $((RANDOM % 5 + 1));  
echo "Processo $$ terminou"'
```

```
mpirun --oversubscribe -np 4 bash -c 'echo "Olá!  
Eu sou o processo $$ e comecei a corrida."; sleep  
0.$((RANDOM % 9 + 1)); echo "--> O processo $$  
terminou!"'
```

```
mpirun --oversubscribe -np 5
```

```
date
```

```
mpirun --oversubscribe -np 4 bash -c 'echo "Olá do  
rank $OMPI_COMM_WORLD_RANK de  
$OMPI_COMM_WORLD_SIZE"'
```

```
Codespaces-0cae63  
@Thaiscangucu → /workspaces/compdist-labestadocompartilhado-swiftnopfler (main) $ mpirun --oversubscribe -np 4 bash -c 'echo "Olá! Eu sou o processo $$"; sleep $((RANDOM % 5 + 1)); echo "Processo $$ terminou"'  
mpirun --oversubscribe -np 4 bash -c 'echo "Olá! Eu sou o processo $$ e comecei a corrida."; sleep 0.$((RANDOM % 9 + 1)); echo "--> O processo $$ terminou!"'  
mpirun --oversubscribe -np 5 date  
mpirun --oversubscribe -np 4 bash -c 'echo "Olá do rank $OMPI_COMM_WORLD_RANK de $OMPI_COMM_WORLD_SIZE"'  
Olá! Eu sou o processo 38254  
Olá! Eu sou o processo 38256  
Olá! Eu sou o processo 38251  
Olá! Eu sou o processo 38252  
Processo 38254 terminou  
Processo 38251 terminou  
Processo 38252 terminou  
Processo 38256 terminou  
Olá! Eu sou o processo 38345 e comecei a corrida.  
Olá! Eu sou o processo 38348 e comecei a corrida.  
Olá! Eu sou o processo 38346 e comecei a corrida.  
Olá! Eu sou o processo 38349 e comecei a corrida.  
--> O processo 38345 terminou!  
--> O processo 38348 terminou!  
--> O processo 38346 terminou!  
Tue Mar 3 23:46:41 UTC 2026  
Tue Mar 3 23:46:41 UTC 2026  
Tue Mar 3 23:46:41 UTC 2026  
Tue Mar 3 23:46:41 UTC 2026  
Olá do rank 0 de 4  
Olá do rank 1 de 4  
Olá do rank 2 de 4  
Olá do rank 3 de 4
```

O que você notou na execução destes comandos?

A ordem de saída dos processos é diferente.

Pesquise as principais variáveis do MPI.

PARTE 2 – Construindo o Cluster Ubuntu

Ainda no Codespaces, crie os arquivos abaixo:

>>> Dockerfile

O Dockerfile define como construir uma imagem (especificação).

Ele responde:

- Qual é o sistema base?
- Quais pacotes devem ser instalados?
- Qual usuário precisa criar?
- Qual comando inicial executar?

```
FROM ubuntu:22.04 RUN
apt update && \
    apt install -y openmpi-bin libopenmpi-dev
opensshserver sudo figlet net-tools iputils-ping && \
useradd -m -s /bin/bash mpiuser && \      echo
"mpiuser:mpi123" | chpasswd && \      adduser mpiuser
sudo && \      mkdir /var/run/ssh
EXPOSE 22
CMD service ssh start && sleep infinity
```

>>> docker-compose.yml

O docker-compose organiza múltiplos containers e irá utilizar a especificação do Dockerfile.

Ele responde:

- Quantos containers desejamos?
- Como eles se chamam?
- Estão na mesma rede?
- Compartilham volume?

- Qual imagem usar?

```
services:
master:
build: .
    container_name: master
hostname: master    networks:
    - mpi_net
    worker1:
build: .
    container_name: worker1
hostname: worker1
networks:    - mpi_net
    worker2:
build: .
    container_name: worker2
hostname: worker2
networks:    - mpi_net
    worker3:
build: .
    container_name: worker3
hostname: worker3
networks:    - mpi_net
    networks:
mpi_net:
```

>>> start_cluster.sh

```
#!/bin/bash docker
compose build docker
compose up -d echo
"Cluster iniciado."
```

No terminal, dar permissões ao arquivo e executar:

```
chmod +x start_cluster.sh ./start_cluster.sh
```

Verificar os containers:

```
docker ps
```

```
@Thaiscangucu → /workspaces/compdist-labestadocompartilhado-swiftnopfler (main) $ chmod +x start_cluster.sh && ./start_cluster.sh
/bin/bash: docker: command not found
@Thaiscangucu → /workspaces/compdist-labestadocompartilhado-swiftnopfler (main) $ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
@Thaiscangucu → /workspaces/compdist-labestadocompartilhado-swiftnopfler (main) $
```

PARTE 3 – Configurando SSH sem senha >>>

setup_ssh.sh

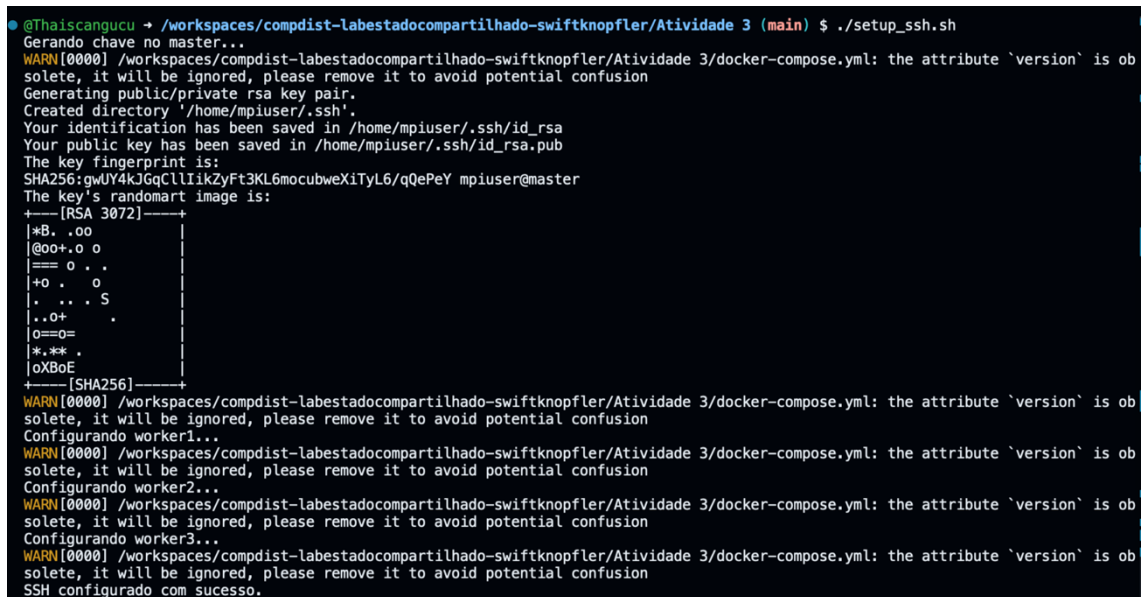
```
#!/bin/bash
echo "Gerando chave no master..."
docker compose exec master sudo -u mpiuser ssh-keygen -t
rsa -N "" -f /home/mpiuser/.ssh/id_rsa

PUBKEY=$(docker compose exec master cat
/home/mpiuser/.ssh/id_rsa.pub)

for node in worker1 worker2 worker3; do
echo "Configurando $node..."
echo "$PUBKEY" | docker compose exec -T $node bash -c "
mkdir -p /home/mpiuser/.ssh && cat >>
/home/mpiuser/.ssh/authorized_keys && chown -R
mpiuser:mpiuser /home/mpiuser/.ssh && chmod 700
/home/mpiuser/.ssh &&
chmod 600 /home/mpiuser/.ssh/authorized_keys
"
done
echo "SSH configurado."
```

Permissão:

```
chmod +x setup_ssh.sh ./setup_ssh.sh
```



```
● @Thaiscangucu → /workspaces/compdist-labestadocompartilhado-swiftnopfler/Atividade 3 (main) $ ./setup_ssh.sh
Gerando chave no master...
WARN[0000] /workspaces/compdist-labestadocompartilhado-swiftnopfler/Atividade 3/docker-compose.yml: the attribute `version` is ob
solete, it will be ignored, please remove it to avoid potential confusion
Generating public/private rsa key pair.
Created directory '/home/mpiuser/.ssh'.
Your identification has been saved in /home/mpiuser/.ssh/id_rsa
Your public key has been saved in /home/mpiuser/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:gwUY4kJGqCllIkZyFt3KL6mocubweXiTyL6/qQePeY mpiuser@master
The key's randomart image is:
+-----[RSA 3072]-----+
|*B. .oo|
|@00+.o o|
|== o . .|
|+o . o|
|. . . S|
|..o+ .|
|o==o|
|*.*.*|
|oXB0E|
+-----[SHA256]-----+
WARN[0000] /workspaces/compdist-labestadocompartilhado-swiftnopfler/Atividade 3/docker-compose.yml: the attribute `version` is ob
solete, it will be ignored, please remove it to avoid potential confusion
Configurando worker1...
WARN[0000] /workspaces/compdist-labestadocompartilhado-swiftnopfler/Atividade 3/docker-compose.yml: the attribute `version` is ob
solete, it will be ignored, please remove it to avoid potential confusion
Configurando worker2...
WARN[0000] /workspaces/compdist-labestadocompartilhado-swiftnopfler/Atividade 3/docker-compose.yml: the attribute `version` is ob
solete, it will be ignored, please remove it to avoid potential confusion
Configurando worker3...
WARN[0000] /workspaces/compdist-labestadocompartilhado-swiftnopfler/Atividade 3/docker-compose.yml: the attribute `version` is ob
solete, it will be ignored, please remove it to avoid potential confusion
SSH configurado com sucesso.
```

PARTE 4 – Execução Distribuída

Entrar no container master:

```
docker compose exec master bash
su - mpiuser sudo apt install
nano
```

Criar o arquivo hostfile

```
nano hosts
```

Conteúdo:

```
master slots=2 worker1
slots=2 worker2
slots=2 worker3
slots=2
```

Desativar a pergunta: Are you sure you want to continue connecting (yes/no/[fingerprint])? do SSH, isso facilita para o master.

```
echo -e "Host worker1 worker2 worker3\n
StrictHostKeyChecking no" >> ~/.ssh/config
chmod 600
```

```
~/.ssh/config
```

Testes o MPI distribuído

```
mpirun --hostfile hosts -np 8 hostname
```

```
mpirun --hostfile hosts -np 8 bash -c 'echo "Rank
$OMPI_COMM_WORLD_RANK rodando em $(hostname) "'
```

```
mpirun --oversubscribe -np 30 --hostfile hosts
hostname
```

```
mpiuser@master:~$ nano hosts
mpiuser@master:~$ echo -e "Host worker1 worker2 worker3 master\n StrictHostKeyChecking no" >> ~/.ssh/config
mpiuser@master:~$ chmod 600 ~/.ssh/config
mpiuser@master:~$ mpirun --hostfile hosts -np 8 hostname
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
master
master
worker1
worker1
worker3
worker3
worker2
worker2
mpiuser@master:~$
mpiuser@master:~$ mpirun --hostfile hosts -np 8 bash -c 'echo "Rank $OMPI_COMM_WORLD_RANK rodando em $(hostname)"'
Rank 1 rodando em master
Rank 0 rodando em master
Rank 6 rodando em worker3
Rank 2 rodando em worker1
Rank 3 rodando em worker1
Rank 7 rodando em worker3
Rank 4 rodando em worker2
Rank 5 rodando em worker2
@Thaiscangucu → /workspaces/compdist-labestadocompartilhado-swiftnopfler/Atividade 3 (main) $ docker compose exec master bash
Rank 4 rodando em worker2
Rank 5 rodando em worker2
mpiuser@master:~$ mpirun --oversubscribe -np 30 --hostfile hosts hostname
master
master
master
master
master
master
worker2
worker3
worker3
worker2
worker2
worker3
worker2
worker3
worker2
worker3
worker2
worker3
worker1
worker1
worker1
worker1
worker1
worker1
worker1
mpiuser@master:~$
```

Dica para Caso o Codespace seja reiniciado

```
docker compose up -d
```

```
docker compose exec master service ssh start
```

Exercícios

Exercício 1 – Figlet distribuído [Imprimir](#)

usando os containers:

"Olá Mack, sou o SEU_NOME"

Cada palavra deve ser executada por um processo diferente.

Executar 5 vezes e registre os resultados. Analise e comente.

Entregar prints e suas análises.

```
@Thaiscangucu → /workspaces/compdist-labestadocompartilhado-swiftnopfler/Atividade 3 (main) $ docker compose exec master bash
```



```
Ola
Meek
THATS
FERREIRA
A A A A A A A A A A
CANGUCU
SOU
```



```
Ola
A - - -
THATS
FERREIRA
CANGUCU
SOU
Meek
```

@k

THATS

NAME A

Mick

FERRERA

SO

@

CANGU

@k

THATS

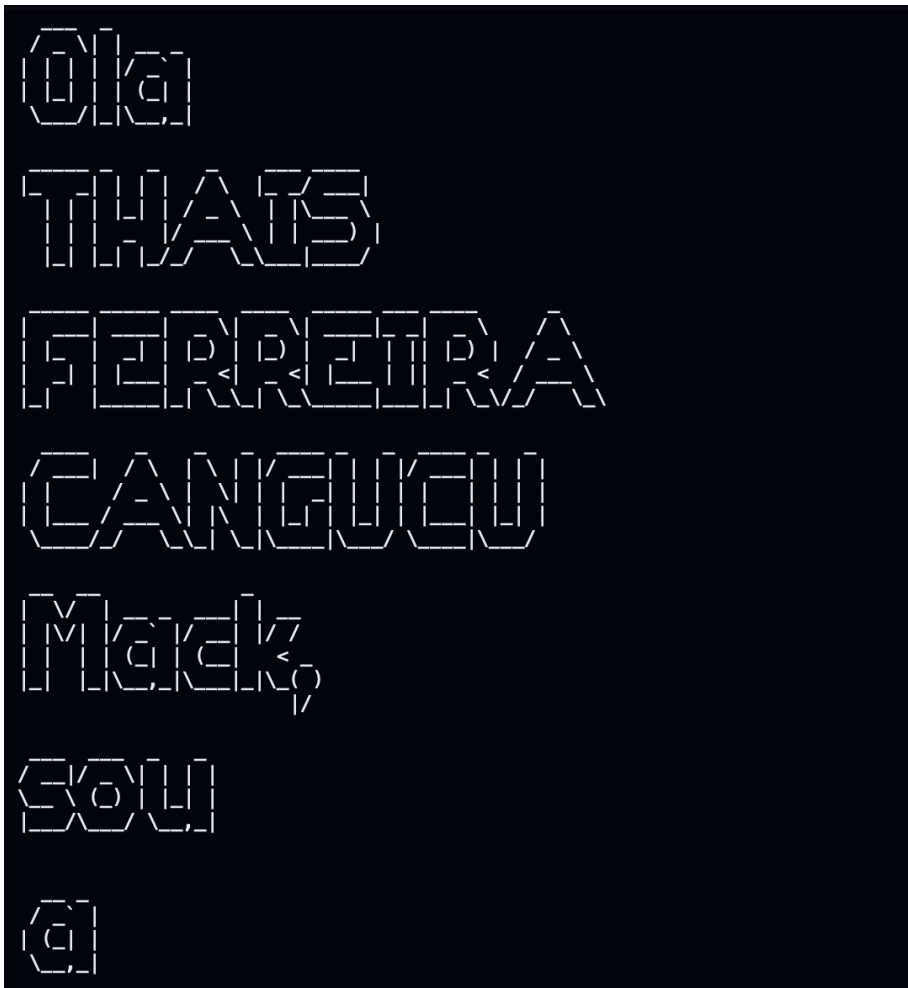
FERRERA

CANGU

SO

Mick

@



As palavras impressas no terminal estão fora de ordem por conta de concorrência dos Workers. Cada palavra foi alocada para um processo diferente e foram rodados simultaneamente. Porém não tem como garantir qual processo termina primeiro por isso a frase sai fora de ordem. A ordem dos ranks depende da latência da rede.

Exercício 2 – Descobrir cores e threads No

Codespace:

nproc lscpu

Executar o programa hostname com mpi nos containers:

- com número máximo de cores físicos
- com número máximo de cores lógicos
- com o dobro do limite de processos (primeiro descubra a capacidade máxima de processos)

Entregar prints e suas análises.

Print do lscpu: Mostrando 1 core físico e 2 threads.

```
mpiuser@master:~$ mpirun --hostfile hosts -np 1 hostname  
[master:00415] plm:rsh: Warning: setpgid(421,421) failed in parent with errno=Permission denied(13)  
master
```

Print do -np 1: Mostrando que o aviso de permissão apareceu, mas o comando rodou.

```
mpiuser@master:~$ mpirun --hostfile hosts -np 2 hostname  
master  
master
```

Print do -np 16: Mostrando a distribuição em massa nos containers.

```
mpiuser@master:~$ mpirun --hostfile hosts -np 16 --oversubscribe hostname  
master  
master  
master  
master  
worker2  
worker2  
worker2  
worker2  
worker1  
worker1  
worker1  
worker1  
worker3  
worker3  
worker3  
worker3
```

O dado do ulimit -u: Que é unlimited.

```
mpiuser@master:~$ ulimit -u  
unlimited
```

Ao executar o comando `ulimit -u`, o sistema retornou `unlimited`. Isso indica que não há um limite de processos imposto. Teoricamente, poderíamos escalar o MPI para vários processos, limitados apenas pelos recursos físicos de RAM e CPU.

Exercício 3 – Cluster soletrando o alfabeto

Fazer o cluster de containers "soletrar" o alfabeto, onde cada processo (de 0 a 25) imprime uma letra diferente em formato de banner (com o figlet). Deve ser exibido o nome do host também. Analise e comente a resultado.

Entregar prints e suas análises.

```
mpiuser@master:~$ mpirun --hostfile hosts -np 26 --oversubscribe bash -c '  
ALFABETO="ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
LETRA=${ALFABETO:$OMPI_COMM_WORLD_RANK:1};  
echo "Processo $OMPI_COMM_WORLD_RANK no host $(hostname):";  
figlet $LETRA  
'
```

Processo 3 no host master:

Processo 4 no host master:

Processo 0 no host master:

Processo 1 no host master:

Processo 2 no host master:

Processo 9 no host worker1:

J

Processo 17 no host worker2:

R

Processo 20 no host worker3:

U

Processo 10 no host worker1:

K

Processo 18 no host worker2:

Processo 21 no host worker3:

V

Processo 11 no host worker1:

L

Processo 14 no host worker2:

@ma1scangucu → /workspaces/compuist-labestadoc

Processo 16 no host worker2:

Q

Processo 24 no host worker3:

Processo 8 no host worker1:

I

S

Processo 25 no host worker3:

Processo 13 no host worker1:

Processo 19 no host worker2:

N

T

Z

Processo 5 no host worker1:

Processo 5 no host master:

V
V
V

Processo 6 no host master:

G
F

Processo 14 no host worker2:

O

Processo 22 no host worker3:

V
V
V
V
V

Processo 15 no host worker2:

D

Processo 23 no host worker3:

X

Processo 12 no host worker1:

M

Processo 7 no host worker1:

L

Utilizamos 26 processos distribuídos em um cluster de 4 containers (8 slots nominais). O uso da flag --oversubscribe foi usado, pois o número de tarefas (26) passa da quantidade de slots disponíveis (8).

O MPI simulou 26 unidades de execução independentes. As letras do alfabeto apareceram de forma desordenada no terminal. Isso porque cada processo concorre pelos recursos de CPU e rede de forma assíncrona. Não tem como garantir ordem de chegada sem um mecanismo de sincronização explícito. É possível perceber que as letras foram processadas alternadamente entre master, worker1, worker2 e worker3, provando a colaboração de todo o cluster.

Exercício 4 – Menor latência

Descobrir qual máquina na rede tem a menor latência (ping mais rápido) em relação à máquina Mestre. Execute um script pelo mpi.

Entregar prints e suas análises.

```
mpiuser@master:~$ mpirun --hostfile hosts -np 4 bash -c '
LATENCIA=$(ping -c 1 master | grep "time=" | awk -F"time=" '{print \$2}');
echo "Sou o host $(hostname) e minha latência para o Master foi: $LATENCIA"
'
Sou o host master e minha latência para o Master foi: 0.044 ms
Sou o host master e minha latência para o Master foi: 0.027 ms
Sou o host worker1 e minha latência para o Master foi: 0.048 ms
Sou o host worker1 e minha latência para o Master foi: 0.036 ms
```

A latência representa o tempo que um pacote de dados leva para ir da origem ao destino e retornar. Em sistemas distribuídos, latências altas podem se tornar o gargalo de performance do cluster. Como os containers compartilham o mesmo hardware físico e rede virtual, os valores observados indicam uma rede extremamente rápida. A máquina com menor latência foi o próprio master e depois workers. Saber qual nó tem menor latência ajuda o escalonador do MPI a decidir onde colocar os processos que precisam trocar dados com mais frequência.

O que deve ser entregue (individual)

Análise dos testes locais da parte 1 juntamente com os questionamentos propostos.

Entregar um print da execução dos containers referente a parte 2

Entregar um print da execução da conf do SSH referente a parte 3

Entregar um print da execução dos Testes do MPI distribuído referente a parte 4

Entregar os exercícios propostos mostrando sua execução juntamente com suas análises